Reproducing and Improving FiDIP for Infant Pose Estimation on SyRIP

Vishnu Purohitham
EECE 7373 Machine Learning with Small Data, Northeastern University, Fall 2026

Build: 2026-11-06 08:01 EST, commit `5650400` (+uncommitted report changes)

Abstract

Infant pose estimation is a small-data problem: privacy limits the number of labelled real images, and adult-trained models transfer poorly to infant bodies and poses. FiDIP [16] fine-tunes a COCO-pretrained DarkPose HRNet-W48 on SyRIP (200 real and 1000 synthetic training images), with a domain classifier that pushes real and synthetic features together. It reports 93.6 AP on SyRIP Test100. We reproduce this result and try to improve on it with an AI coding agent on Northeastern's Explorer cluster. Every evaluation condition is stated, and every model is scored by one audited evaluator.

The released FiDIP checkpoint scores 93.6 AP on Test100. The released training loop selects its checkpoint on Test100. We instead select all models on Val400, the 397 real validate500 images that neither belong to nor duplicate Test100, and evaluate Test100 only for pre-registered configurations. Retraining FiDIP gives 93.6 ± 0.4 AP over 3 seeds, against **TBD** $\pm$ **TBD** AP without domain adaptation.

Checking the data found issues that change the numbers. validate500 ships a constant placeholder object area. An annotation with id 0 in Test100 is never matched by pycocotools, which caps recall at 0.99 in our numbers and the published ones alike. Five Test100 images duplicate training images. And the training file the README points to carries another dataset's labels. The released checkpoint and our retrains reproduce 93.6, but FiDIP's gain from domain adaptation over plain fine-tuning does not reproduce.

A controlled backbone and resolution study in MMPose leads to our pre-registered configuration, ViTPose-H 256 (arm B). It scores **TBD** $\pm$ **TBD** AP on Test100, **TBD** AP from the released checkpoint (95% CI [**TBD**, **TBD**]). Every run, decision and failure is logged in the repository.

1 Introduction

The problem. Long-term observation of infant posture and movement can support early developmental risk assessment. Automating it starts with 2D keypoint estimation from ordinary RGB images [16]. Infants make this a small-data problem in two ways. Privacy limits the number of labelled real images. And infants differ from the adults in COCO [20] in limb proportions, pose repertoire and context, so adult-trained estimators degrade on infant-specific poses [16].

FiDIP. FiDIP (Fine-tuned Domain-adapted Infant Pose) [16] addresses both. It fine-tunes a COCO-pretrained pose network on SyRIP [3], a mix of 200 real web images and 1000 synthetic renders, while a domain classifier pushes the features of real and synthetic images together. Its strongest variant, DarkPose [28] with an HRNet-W48 backbone [24], reports 93.6 AP on SyRIP Test100 [2]. As far as we could establish (Section 2.5), this is still the best published number under that protocol.

Goals. This report documents a hands-on reproduction of FiDIP and an attempt to improve on it, carried out with an AI coding agent on Northeastern's Explorer cluster. We had four goals:

(G1) Reproduce the published baseline, from the released checkpoints and by retraining.
(G2) Make the evaluation conditions explicit, and identical for every number we compare.
(G3) Improve Test100 AP on the same split, without using Test100 for any decision.
(G4) Keep the work reproducible and the agentic process auditable.

Findings.

- **Reproduction** (Section 4). The released FiDIP checkpoint scores 93.6 AP on Test100 with our evaluator, matching the published 93.6 exactly. Retraining with the released recipe gives 93.6 ± 0.4 AP with domain adaptation and **TBD** $\pm$ **TBD** AP without it (3 seeds each). The paper's +0.9 AP from domain adaptation does not reproduce.
- **Protocol** (Sections 2.3 and 3.3). Run literally, the README training command fine-tunes from random weights with the adaptation weight $\lambda = 0$, and the training loop keeps the epoch that is best on Test100. We select every model on Val400, the 397 validate500 images that neither belong to nor duplicate Test100, and pre-register each configuration before its single Test100 evaluation.
- **Data and evaluation conditions** (Sections 3.3 and 3.4). Hashing, near-duplicate search and label/image checks found six issues that would silently bias results (Table 3):
  - placeholder areas;
  - an annotation with id 0 that pycocotools never matches (+1.4 AP for the released model);
  - five test images that duplicate training images;
  - three Val400 candidates that duplicate test images;
  - mislabelled synthetic images in the README's training file;
  - unannotated duplicates in the training folder.

  Claims must hold under both matching protocols.
- **Improvement** (Section 5). We ran a controlled backbone and input-resolution comparison under one MMPose recipe, plus ablations, all selected on Val400. The pre-registered winner, ViTPose-H 256 (arm B), scores **TBD** $\pm$ **TBD** AP on Test100. That is **TBD** AP from the released checkpoint, with a paired-bootstrap 95% CI of [**TBD**, **TBD**].
- **Process** (Sections 8 and 9). The agent kept `SKILL.md`, `AGENT.md` and `MANIFEST.md` current during the work, not after it, and every reported number traces to a run and a commit.

Roadmap. Section 2 covers FiDIP, SyRIP and the gaps between paper and code. Section 3 covers the workflow, data, protocol and evaluator. Sections 4 to 6 report the reproduction, the improvement study and the analysis. Sections 7 to 9 cover limitations, lessons and AI-tool use. The appendix gives commands, details, all attempts, provenance and a rubric map.

2 Background

2.1 Top-down pose estimation and its metric

Models. A top-down estimator crops each person box to a fixed input size (e.g., 384×288), predicts one heatmap per keypoint at 1/4 resolution, and decodes coordinates. SimpleBaseline [26] adds deconvolutions to a ResNet. HRNet [24] keeps a high-resolution branch (width 48 in W48) throughout. DARK [28] decodes sub-pixel maxima from the smoothed log-heatmap, and UDP [15] removes

biases in the resize/flip transforms and encoding. ViTPose [27] uses a plain Vision Transformer. Flip testing averages the heatmaps of the image and its mirror.

Metric. COCO evaluation [1, 20] scores a prediction against a ground-truth (GT) instance by the object keypoint similarity

$$\mathrm{OKS} = \frac{\sum_i \exp\left(-d_i^2/(2\,s^2\kappa_i^2)\right)\,\delta(v_i > 0)}{\sum_i \delta(v_i > 0)}, \qquad \kappa_i = 2\sigma_i, \tag{1}$$

where d_i is the pixel error of keypoint i, s^2 is the GT object *area*, σ_i are the 17 COCO constants, and v_i is the visibility flag. AP averages, over OKS thresholds $t \in \{0.50, \ldots, 0.95\}$, the area under the precision–recall curve of detections ranked by score. AP_{50} and AP_{75} are single thresholds, and AR is the corresponding mean recall. SyRIP is evaluated with GT person boxes and one infant per image, so AP depends only on the per-image OKS and on how the instance scores rank the images.

2.2 The SyRIP dataset

Images. SyRIP [3, 16] has 700 real images from YouTube and Google Images; the README says the 200 real training images show more than 100 infants. It also has 1000 synthetic images, rendered from the SMIL infant body model [13]. To make each one, SMIL is fitted to a real infant's 2D pose, the pose is perturbed, and the body is rendered with random textures (more than 12), lighting, camera and background [16].

Splits. Training uses all 200 real and 1000 synthetic images. validate500 (the paper's *Test500*) holds 500 real images, and its 100 most complex poses form validate100 (*Test100*). Annotations use COCO's 17 keypoints. Section 3.3 describes the overlaps we found between these splits.

Other infant data. MINI-RGBD [14], the only earlier public infant dataset, has 12 synthetic sequences of simple poses. A password-protected *expanded* SyRIP adds 400 real training images; Section 7 explains why we do not use it.

2.3 FiDIP, as published and as released

As published. FiDIP [16] attaches a binary real/synthetic domain classifier D to the feature extractor of a COCO-pretrained pose network f. Training has two phases:

1. *Initialisation:* D learns to separate adult real images (1904 from COCO val2017) from adult synthetic ones (2000 from SURREAL [25]), with f frozen.
2. *Alternating training:* each iteration has two stages. Stage I updates D with the binary cross-entropy $\mathcal{L}_D$. Stage II updates f, with D fixed, to minimise

$$\mathcal{L}(\theta_f, \theta_y, \theta_d) = \mathcal{L}_P(\theta_f, \theta_y) - \lambda\,\mathcal{L}_D(\theta_f, \theta_d), \tag{2}$$

   where $\mathcal{L}_P$ is the heatmap MSE.

The $-\lambda\mathcal{L}_D$ term rewards features that D cannot separate by domain. This is the domain-adversarial idea of Ganin and Lempitsky [10], Ganin et al. [11], which the paper describes as a gradient reversal layer (GRL).

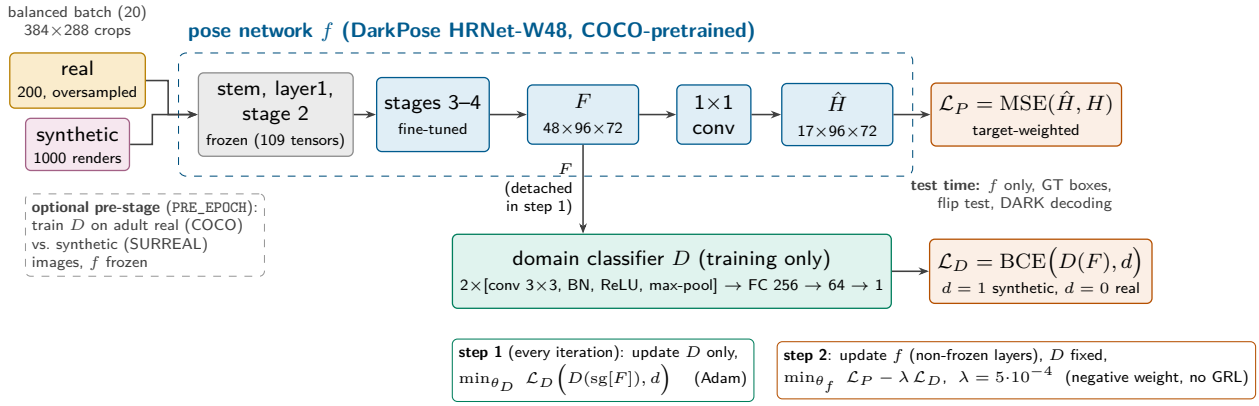


Figure 1: FiDIP as released (DarkPose HRNet-W48), from `lib/models/adaptive_pose_hrnet.py` and `lib/core/function.py`. Step 1 of each iteration updates D on detached stage-4 features. Step 2 updates the pose network with $\mathcal{L}_P - \lambda\mathcal{L}_D$, implemented as a negative loss weight. Only the pose network runs at test time.

As released. We audited the official code [2] at commit `a76b4f4`. Figure 1 shows the released HRNet variant, and Table 1 compares the paper, the released code and our runs. Three differences decide what a reproduction measures:

(i) The infant config sets $\lambda = 0$, so running it literally is plain fine-tuning.

(ii) The config loads no checkpoint: `MODEL.PRETRAINED` is empty, so without a `TRAIN.CHECKPOINT` override training starts from random weights.

(iii) After every epoch the training loop evaluates on `validate_infant` (Test100) and keeps the best epoch as `model_best`, which is selection on the test set.

2.4 Published results and where the headroom is

Table 2 copies the published Test100 numbers. With one infant and one detection per image, the true positives at a stricter OKS threshold are a subset of those at a looser one, so AP_t cannot increase with t. DarkPose + FiDIP reports $\mathrm{AP}_{50} = \mathrm{AP}_{75} = 98.5$, so all six thresholds from 0.50 to 0.75 score 98.5. The four strict thresholds then average

$$\mathrm{AP}_{80:95} = (10 \times 93.6 - 6 \times 98.5)/4 \approx 86.3.$$

The same calculation gives 84.0 for DarkPose + Finetune (92.7 AP, also 98.5/98.5). FiDIP's 0.9-point gain therefore lies entirely at strict thresholds, and so must any further gain. Section 3.4 shows that 98.5 is essentially a perfect score at loose thresholds on this test set.

2.5 Related work on SyRIP

We found no later paper that beats 93.6 on Test100 with GT boxes. Later work on SyRIP uses other protocols:

- zero-shot adult models with *detected* boxes on the 500 real test images (ViTPose best, at 41.9 AP) [9];
- unsupervised adaptation measured by PCK [6];
- 3D pose on Test100 [17, 29];
- cross-dataset checks [12, 19].

Table 1: FiDIP as described in the paper (arXiv v5), as released (code and `w48_384x288_adam_lr1e-3_infant.yaml` at `a76b4f4`), and as we ran it.

Aspect	Paper [16]	Released code / config [2]	Our reproduction
Initialisation of f	COCO-pretrained DarkPose	none unless `TRAIN.CHECKPOINT` is passed (random init)	COCO DarkPose `w48_384x288.pth`
λ	$5{\cdot}10^{-4}$	0 (infant yaml); $5{\cdot}10^{-4}$ only in `_custom.yaml`	$5{\cdot}10^{-4}$; 0 as the fine-tune ablation
Optimiser, LR	Adam, 10^{-3}	Adam, 10^{-4}, constant (LR steps at epochs 40 and 200 are never reached); 10^{-3} in `_custom.yaml`	released yaml; paper recipe as one extra attempt
Epochs / batch	100 / 64	20 / 20	released yaml
Adversarial update	gradient reversal layer	alternating updates with a negative loss weight; no GRL module	as released
Domain classifier	three FC layers	2×[conv3×3, BN, ReLU, max-pool] + FC 256–64–1 on the 48-channel stage-4 high-resolution branch	as released
Classifier pre-stage	1 epoch, batch 128; 1904 COCO vs. 2000 SURREAL adults	`PRE_EPOCH` 0 (infant yaml), 10 (`_custom.yaml`); the adult set (2029 real + 2028 synthetic) is said to be "on request" but ships in `syrip_for_train.zip`	`PRE_EPOCH` 10 on the shipped adult set
Frozen layers	Res1–3 (ResNet ablation)	first 109 parameter tensors (stem, `layer1`, stage 2)	as released
Real/synthetic balance	domain-dependent loss scale $S(I_i)$	balanced sampler (200 real oversampled to 1000, interleaved, no reshuffle between epochs); unweighted loss	as released
Checkpoint selection	not stated	best Test100 AP, checked every epoch	best Val400 AP epoch

Table 2: Published SyRIP Test100 results [2, 16]: COCO-pretrained ("Base"), fine-tuned on SyRIP, and FiDIP. The published baseline of this report is DarkPose + FiDIP, which also reports $AP_{50} = AP_{75} = 98.5$.

Method	Backbone	Input	Base AP	+ Finetune AP	+ FiDIP AP	AR (FiDIP)
SimpleBaseline	ResNet-50	384×288	82.4	90.1	91.1	92.6
DarkPose	HRNet-W48	384×288	88.5	92.7	**93.6**	94.6
Pose-MobileNet	MobileNetV2 [23]	224×224	46.5	78.9	79.2	84.1

AggPose [7] and InfantNet [18] use their own, larger datasets.

3 Implementation

3.1 Compute and agentic workflow

Setup. All training and evaluation ran under Slurm on Northeastern's Explorer cluster [22]. Claude Code [5], running Claude Opus 5.5, acted as an orchestrator (Figure 2). After the student approved a written plan, it ran four parallel workers with self-contained briefs: repro, improve, analysis and report.

Records. The orchestrator keeps `SKILL.md` (cluster operation), `AGENT.md` (the lab notebook, with dated decisions D1, D2, ...) and `MANIFEST.md` (the data, checkpoints and configs behind every number) current during the work. Workers log details in `notebook/<stream>.md`, and every run

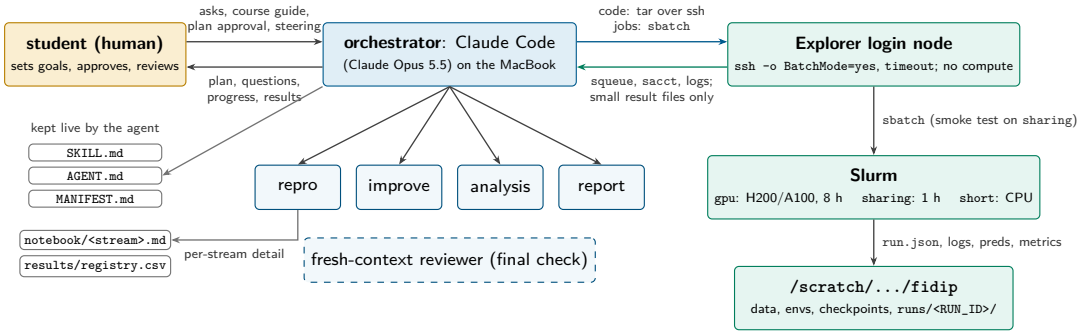


Figure 2: Agentic workflow. The student sets goals, supplies the course material and approves the plan. The orchestrator keeps the three live files current and delegates to parallel workers. All compute goes through Slurm, and only small artifacts return to the laptop.

is registered in `results/registry.csv` with a `run.json` that records the commit and tree SHAs, Slurm job, GPU, seed and checkpoint SHA256.

Rules. The rules in `CLAUDE.md` bind every agent: no compute on login nodes, no selection on Test100, one evaluator for every number, secrets only in a git-ignored `.env`, and timestamps from `date`. Appendix I.1 gives the job loop (BatchMode SSH, tar-over-ssh sync, `sbatch`, polling, small-file pull-back).

3.2 Environments and compatibility patches

The upstream `fidip_env.yml` cannot be built (it pins wheels that do not exist for Python 3.12). We built a pip-only FiDIP environment with the yml's versions, an MMPose 1.3.2 environment [21], and a CPU analysis environment with the same NumPy and pycocotools pins (Appendix I.2). Ten small upstream patches, one commit each, are listed in Appendix B; the most consequential, the README training command, needs only command-line overrides. The course guide's `gpu_test.py`, `tools/demo.py` and `tools/streamlit_demo.py` do not exist upstream. We wrote them from the guide's documented interface.

3.3 Data preparation and the split protocol

Data. Both public SyRIP archives were downloaded and checksummed, and extracted in a Slurm job (Appendix I.3). File hashes, a flip-aware perceptual near-duplicate search (`scripts/near_dups.py`) and label/image consistency checks found six problems, summarised in Table 3. We use `SyRIP.zip`'s images and official `200R_1000S` labels throughout (D17). We rebuilt validate500 with area $= w \cdot h$ (D8) and keep Test100 exactly as shipped.

Splits. **Val400** is validate500 ids 100–499 minus three duplicates of Test100 images, 397 images in all (D15). It is disjoint from Test100 and almost twice the size of the real training set. Unlike cross-validation, it leaves the training data identical to the paper's. Its costs are that Test500 is never reported as a clean test set, and that Val400 lacks the complex-pose selection of Test100 (Section 6.1).

Protocol (D1). Every choice of epoch, hyperparameter, backbone or recipe is made on Val400 (Figure 3). A configuration reaches Test100 only after a commit lists it, with its Val400 evidence and checkpoint rule, in `results/PREREGISTRATION.md`. The final-evaluation script refuses unregistered

Table 3: Dataset-hygiene findings. Effects are on the released FiDIP model; Table 13 has the full comparison.

Finding	Effect	Our handling
validate500 and train JSON carry a constant placeholder `area` (validate100 uses $w \cdot h$)	Val400 AP 98.9 instead of 90.4	recompute $w \cdot h$ (D8)
Test100 annotation id 0 is never matched by pycocotools	ΔAP +1.4 (fixed − paper); range 92.98–94.07 over test0's rank	two protocols; claims must hold under both (D10, D13)
Test100 images 20, 42, 46, 50 are byte-identical to training images; test7 is a near-copy	Test100-clean ΔAP −0.8	keep as shipped; report Test100-clean (D16)
3 validate500 images duplicate Test100 images	selection-set leakage	removed: Val400 = 397 (D15)
`syrip_for_train.zip`: MINI-RGBD labels on all 1000 synthetic images; its `test11.jpg` is a copy of `test24.jpg`	R020 Val400 −1.2; test11: −1.5 AP	use `SyRIP.zip 200R_1000S` (D17)
300 unannotated real images in `train_infant/`, 199 identical to validate500 images	none if unused	use only annotated images

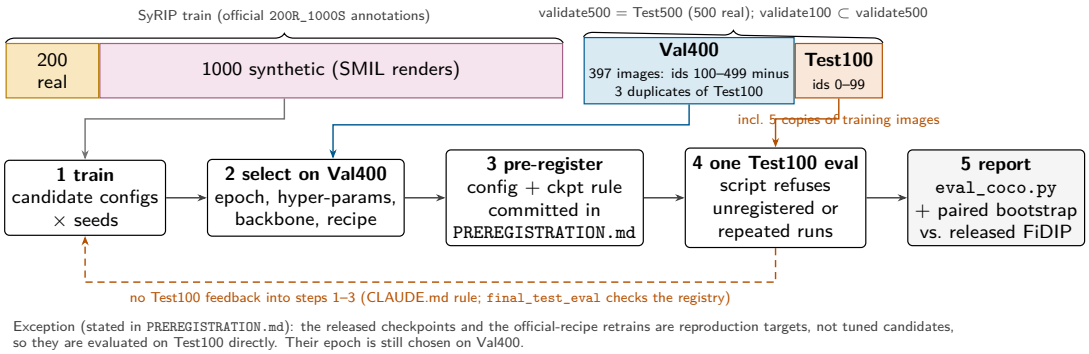


Figure 3: Split and selection protocol (bar widths proportional to image counts). Test100 is read once per pre-registered configuration, and nothing learned from it flows back into training or selection.

or repeated runs and verifies checkpoint SHA256s. The released checkpoints and the official-recipe retrains are reproduction targets, so they are evaluated on Test100 directly; their epochs are still chosen on Val400.

3.4 One evaluator for every number

Pipeline. Every metric comes from `eval/eval_coco.py` (v1.0) applied to a prediction JSON, through one path for both frameworks:

1. a loader that accepts twelve prediction encodings;
2. one rescoring rule, score = mean($c_k \mid c_k > 0.2$), which is upstream's rule and MMPose's `bbox_keypoint` mode with GT boxes, so the image ranking and hence AP stay comparable;
3. a port of upstream OKS-NMS;
4. pycocotools 2.0.8 `COCOeval` [1], with the COCO sigmas and the GT area.

It also writes per-threshold AP, $AP_{80:95}$, per-image OKS, and per-keypoint errors normalised by $\sqrt{wh}$ of the GT box, with PCK.

The annotation-id-0 artifact (D10). In Test100, annotation ids equal image ids, so `test0.jpg` carries id 0. `COCOeval.evaluateImg` stores the matched GT id in `dtMatches`, and `accumulate`

treats a stored 0 as "unmatched". The infant in `test0.jpg` can therefore never be a true positive, and its detection always counts as a false positive. This caps AR at 0.99, and even a perfect model's AP at every threshold is

$$\frac{r + 0.99\,(100 - r)}{101} \in [0.980,\ 0.990],$$

where r is the rank of test0's score (unit-tested). The published $\mathrm{AP}_{50} = \mathrm{AP}_{75} = 98.5$ falls inside this band, so on the 99 matchable images the published models are already perfect at loose thresholds. The near-constant $\mathrm{AR}_{50} = 99.0$ of the upstream table has the same cause.

The size of this penalty depends only on where test0's score ranks: between 0.9 and 2.0 AP at realistic accuracy. Our headline `paper` protocol keeps the ids as shipped, so it stays comparable with 93.6. A `fixed` protocol shifts the ids off 0. An improvement claim counts only if its paired CI excludes 0 under *both* protocols, and every model's test0 rank is reported (D13). On Val400, which has no id 0, the two protocols coincide. This is a property of the evaluation tooling, not an error in the published work: anyone scoring this file with pycocotools gets the same cap.

MMPose's evaluator. MMPose's `CocoMetric` uses xtcocotools [4], which marks unmatched detections with -1. Its printed Test100 AP therefore equals our `fixed` protocol, +1.4 AP away from `paper` on identical predictions. Comparing it with 93.6 would overstate any gain, which is why only `eval_coco.py` numbers are compared (D2).

Verification and statistics.

- *Tests.* 69 unit tests (Appendix C).
- *Cross-check.* Our evaluator agrees with upstream's printed AP on all 21 run/split pairs, to within 0.0005 AP.
- *Uncertainty.* `eval/bootstrap_ci.py` resamples the 100 test images ($B = 10000$) and recomputes AP exactly. It reports paired 95% percentile CIs of seed-averaged AP differences, and Holm-adjusted one-sided p-values [8].

4 Reproduction results

4.1 Released checkpoints

We evaluated `hrnet_fidip.pth` (DarkPose + FiDIP) and `mobile_fidip.pth` (Pose-MobileNet + FiDIP) under their released test settings: GT boxes, flip test and DARK post-processing. Their predictions were then scored by `eval_coco.py`; Table 4, upper block, gives the results.

HRNet. The released HRNet checkpoint reproduces the published numbers *exactly*:

- 93.6 AP (published 93.6);
- $\mathrm{AP}_{50}/\mathrm{AP}_{75}$ 98.5/98.5 (published 98.5/98.5);
- AR 94.6 (published 94.6).

Upstream's own test script prints the same AP (93.6). Over all 21 run/split pairs, our evaluator agrees with upstream's printout to within 0.0005 AP. The same predictions score 95.0 AP under the `fixed` protocol, 92.8 AP on Test100-clean, and 90.4 AP on Val400.

MobileNet. The released Pose-MobileNet scores 79.1 AP, against a published 79.2.

Table 4: Reproduction on SyRIP Test100 (`paper` protocol unless marked; GT boxes; flip test) and on Val400. *Upper block:* released checkpoints, scored by `eval_coco.py`. *Lower block:* our retrains of the released recipe from COCO DarkPose weights, mean ± s.d. over seeds, at the Val400-selected epoch and at the final epoch. "Publ." is the published AP [2, 16]; "clean" is Test100-clean.

		SyRIP Test100									Val400
Model	Weights / runs	Publ. AP	AP	AP_{50}	AP_{75}	$AP_{80:95}$	AR	AP^{fixed}	AP^{clean}	$AP^{clean,fixed}$	AP
DarkPose W48 (COCO only)	`w48_384x288.pth`	88.5	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
DarkPose + FiDIP	`hrnet_fidip.pth`	93.6	93.6	98.5	98.5	86.2	94.6	95.0	92.8	94.9	90.4
Pose-MobileNet + FiDIP	`mobile_fidip.pth`	79.2	79.1	99.0	89.3	54.7	84.1	79.8	77.9	78.7	68.2
DarkPose + Finetune (λ=0)	retrain, TBDseeds, best-Val400 epoch	92.7	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
	retrain, TBDseeds, final epoch		TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
DarkPose + FiDIP ($\lambda=5{\cdot}10^{-4}$)	retrain, 3 seeds, best-Val400 epoch	93.6	93.6 ±0.4	98.5	98.5	86.2	94.6	95.1 ±0.5	92.8 ±0.4	95.0 ±0.4	90.5 ±0.1
	retrain, 3 seeds, final epoch		93.4 ±0.5	98.5	98.5	85.7	94.5	94.9 ±0.5	92.7 ±0.4	94.8 ±0.4	90.4 ±0.3

Val400 versus Test100. Val400 is lower than Test100 for every model we evaluated on both (Section 6.1). Part of the gap comes from validate500's tighter boxes: on the same 100 images, swapping in validate500's boxes lowers the released model from 93.6 to 92.0 AP.

4.2 Retraining FiDIP and the domain-adaptation ablation

Recipe. We retrained with the released code and yaml plus two command-line overrides (D5): `TRAIN.CHECKPOINT`, set to the COCO DarkPose weights, and `TRAIN.LAMBDA`, set to $5{\cdot}10^{-4}$.

- The classifier pre-stage runs for 10 epochs (as in `_custom.yaml`) on the shipped adult set.
- After each epoch the model is evaluated on Val400, and the best epoch is kept.
- The same recipe with $\lambda = 0.0$ isolates domain adaptation.
- Each arm has 3 seeds.

FiDIP reproduces. The FiDIP retrains score 93.6 ± 0.4 AP on Test100 at the Val400-selected epoch, and 93.4 ± 0.5 at the final epoch. On Val400 they score 90.5 ± 0.1. Paired with the released checkpoint on the same images, the difference is -0.05 AP, with a 95% CI of [-0.67, 0.65]. Under `fixed` it is 0.12 [-0.61, 0.72] (Table 6). **The training pipeline therefore reproduces the published 93.6, under a stricter protocol that never selects on the test set.**

The domain-adaptation gain does not reproduce. With $\lambda = 0$ the same recipe scores **TBD** ± **TBD** AP on Test100, 93.7 ± 0.1 at the final epoch, and **TBD** ± **TBD** on Val400. FiDIP minus fine-tuning is **TBD** AP, 95% CI [**TBD**, **TBD**]. The paper's +0.9 AP from domain adaptation (92.7 to 93.6) is therefore *not* reproduced. The adversarial term neither helps nor hurts measurably at $n = 100$.

Our $\lambda = 0$ runs still contain the classifier pre-stage, so we also ran the paper-faithful plain fine-tune without it (`PRE_EPOCH` 0, R021–R023, D21). It scores **TBD** ± **TBD** AP on Test100 and **TBD** ± **TBD** on Val400.

Other reproduction attempts (Val400, one seed; all in Table 14).

- **λ sweep.** Accuracy falls monotonically as λ grows: 10^{-4} scores 91.0, $2{\cdot}10^{-3}$ 89.7, and 10^{-2} 85.8 (83.8 by the last epoch), against 90.5 ± 0.1 at $5{\cdot}10^{-4}$.
- **The paper's recipe** (Adam 10^{-3}, 100 epochs, batch 64; R019) peaks at 90.7 at epoch 10, then overfits, ending at 89.8. It explains no gap: the shipped yaml already reproduces 93.6.
- **The README-literal training labels** (R020, the MINI-RGBD labels on the synthetic images) score 89.3, which is −1.2 AP against the retrains with the official labels.



Figure 4: Retrain curves: per-epoch training loss, Val400 loss and Val400 AP for all seeds. Test100 was never evaluated per epoch.

4.3 Training dynamics: over- and underfitting

Every run logs the training loss, the Val400 loss and Val400 AP per epoch (Figure 4). For the FiDIP retrains, the training loss keeps falling, while the Val400 loss flattens after about epoch 6. Val400 AP, however, still rises: the selected epochs are 18–20 of 20 for $\lambda = 5{\cdot}10^{-4}$ and 16–20 for $\lambda = 0$. There is thus no AP overfitting within the released 20-epoch schedule.

A longer schedule does not help either. The paper's 10× learning rate reaches the same Val400 level by epoch 10 and then overfits, as described above. A late LR decay looks like the better lever than more epochs. The data invite overfitting in two ways: each of the 200 real images is seen about five times per epoch, and the balanced sampler repeats the same batch order every epoch. The released recipe counteracts this with a low constant LR and frozen early layers.

5 Improving on FiDIP

5.1 Hypothesis and method

Hypothesis. The published model is already essentially perfect at OKS 0.50–0.75 on the 99 matchable images (Sections 2.4 and 3.4), so any gain must come from more precise localisation, as measured by $AP_{80:95}$. Two standard levers act on precision: backbone capacity with stronger pre-training, and input resolution. We test both with ViTPose [27]. In the MMPose model zoo [21], its COCO checkpoints score 78.2 AP (L) and 78.8 AP (H), against 77.2 AP for HRNet-W48 + DARK at 384×288

(COCO val2017, detected boxes).

One recipe for every arm. All improvement runs use MMPose 1.3.2.

- The control is HRNet-W48 + DARK at 384×288, the MMPose counterpart of FiDIP's network. The treatments are ViTPose-L and ViTPose-H at 256×192 and 384×288.
- Shared by all arms:
  - the training data (200 real + 1000 synthetic, official labels);
  - FiDIP's 1:1 real:synthetic mix (each real image five times per 2000-sample epoch);
  - LR 10^{-4}, 30 epochs with steps at 24 and 29, and MMPose's COCO augmentation;
  - GT boxes, flip test, and per-epoch Val400 checkpoint selection.
- What differs is only what each family needs: Adam versus AdamW with layer-wise decay and drop-path, the batch size, and the codec (DARK versus UDP).

Table 8 lists the resolved settings. The only pose data besides SyRIP are the COCO checkpoints; we excluded models whose pre-training adds other pose datasets.

5.2 Screening and ablations on Val400

Screening (one seed each, Val400 only; Table 11 in Appendix J).

- Fine-tuning on SyRIP lifts every COCO model well above its zero-shot Val400 AP, which ranges from 86.5 (HRNet) to 88.8 (ViTPose-H 256).
- The control reaches 91.5.
- ViTPose-L reaches 92.8 at 256×192 and 92.8 at 384×288.
- ViTPose-H reaches 93.2 and 93.3 at the same two sizes.
- The learning-rate sweeps are flat (HRNet at $2{\cdot}10^{-4}$ and $5{\cdot}10^{-5}$: 91.4 and 91.7), so single-seed noise is about ±0.2.

Against that noise, *capacity* matters (ViTPose-H beats ViTPose-L by about 2–3× the noise), and *resolution* does not (384×288 versus 256×192 is within noise for both sizes). We therefore chose ViTPose-H at 256×192 over 384×288, which would have cost about twice the compute and memory (`results/PREREGISTRATION.md`).

Ablations (ViTPose-L 256 unless noted; Table 12).

- Synthetic data still helps a strong backbone: training on the 200 real images alone gives 92.2, against 92.8 with synthetic data.
- Rotation up to ±180° gives 92.6, no gain.
- Flip testing adds 0.2–0.7 AP on every model.
- The FiDIP weight λ behaves as described in Section 4.2.

5.3 All attempts

Table 14 (Appendix J) lists all 29 attempts with their accuracy, including those that did not help: the reproduction variants, the domain-adaptation tweaks, and every backbone, resolution, augmentation and recipe change. Test100 is shown only for reproduction targets and pre-registered runs.

Table 5: Final Test100 comparison (`paper` protocol). ΔAP is paired with the released checkpoint on the same images (95% percentile bootstrap CI, $B = 10000$); p is one-sided and Holm-adjusted. The published row is copied from [2].

Model	Seeds	AP	AP_{50}	AP_{75}	$AP_{80:95}$	AR	ΔAP vs. released [95% CI]	p
DarkPose + FiDIP, published	—	93.6	98.5	98.5	—	94.6	—	—
DarkPose + FiDIP, released ckpt	—	93.6	98.5	98.5	86.2	94.6	0 (reference)	—
DarkPose + FiDIP, our retrain	3	93.6 ±0.4	98.5	98.5	86.2	94.6	−0.1 [−0.7, +0.6]	0.532
Best configuration (ViTPose-H 256 (arm B))	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD

Table 6: Paired image-level bootstrap contrasts on Test100. "Robust" means the 95% CI excludes 0 under both `paper` and `fixed` (D13). W/T/L counts per-image OKS wins, ties and losses.

Contrast	ΔAP	95% CI	p	p_{Holm}	fixed: Δ [CI]	robust	W/T/L
FiDIP retrain vs released	−0.1	[−0.7, +0.6]	0.532	0.532	+0.1 [−0.6, +0.7]	no	61/0/39
FiDIP retrain vs released (Test100-clean)	+0.0	[−0.7, +0.7]	0.482	0.482	+0.1 [−0.6, +0.8]	no	58/0/37

5.4 Pre-registered confirmatory test

Design (D20). After screening, and before any MMPose run touched Test100, we registered the following at commit `72f9449`:

- **Arms.** Arm A is the HRNet-W48 control (R131–R135) and arm B is ViTPose-H 256 (R136–R140), with the same recipe. Both use fresh seeds s2–s6, so the screening winner's seed cannot inflate the estimate; the screening seeds R101 and R105 are reported separately.
- **Secondary claim.** An arm-B ensemble (R141).
- **Contrasts.** C1: arm B versus the released checkpoint (primary). C2: B versus A. C3: B versus our FiDIP retrains. C4: the ensemble versus the released checkpoint. All are paired image bootstraps with $B = 10000$, Holm-corrected.
- **Decision rule (H1).** Arm B's mean Test100 AP exceeds 93.6, *and* the C1 CI excludes 0 under both protocols.

Checkpoint SHA256s were verified before each single evaluation. GPUs were a mix of H200, A100, L40S and V100 (D22), recorded per run.

Results (Tables 5 and 6).

- Arm B scores **TBD** ± **TBD** AP on Test100 over 5 seeds, and arm A **TBD** ± **TBD**.
- C1, arm B minus the released checkpoint: **TBD**, 95% CI [**TBD**, **TBD**], or **TBD** [**TBD**, **TBD**] under `fixed`.
- C2, arm B minus arm A: **TBD** [**TBD**, **TBD**].
- C3, arm B minus the retrains: **TBD** [**TBD**, **TBD**].
- C4, the ensemble (R141): **TBD** AP, a difference from the released checkpoint of [**TBD**, **TBD**].

TBD

Published baseline vs. our result, and the exact evaluation conditions

Published baseline. DarkPose + FiDIP, HRNet-W48, 384×288: **93.6 AP** on SyRIP Test100 (AP_{50} 98.5, AP_{75} 98.5, AR 94.6) [2, 16].
Reproduced. The released `hrnet_fidip.pth` scores 93.6 AP under our evaluator (95.0 under `fixed`). Our own retrain scores 93.6 ± 0.4 (Section 4).
Our result. ViTPose-H 256 (arm B): **TBD** ± **TBD** **AP** on Test100, mean ± s.d. over 5 fresh

seeds, `paper` protocol.

- Against the released checkpoint: ΔAP = **TBD**, 95% CI [**TBD**, **TBD**]; under `fixed`, **TBD** [**TBD**, **TBD**].
- Robust under both protocols: **TBD**. Against the published 93.6: **TBD**.
- $AP_{80:95}$ **TBD**; Test100-clean **TBD** AP.

Evaluation conditions, identical for every compared row:

- *Test set:* SyRIP `validate100` (= Test100), 100 real images exactly as distributed. Boxes, areas and annotation ids are unchanged, and the five copies of training images are kept, as in the published evaluation. Test100-clean (95 images) is a sensitivity check.
- *Training data:* 200 real + 1000 synthetic images with the official `200R_1000S` labels. No Val400 or Test100 image was added.
- *Inference:* GT person boxes, flip test yes, input 256×192, and one model per seed. The ensemble is reported separately.
- *Metric:* `eval/eval_coco.py` v1.0, i.e. pycocotools 2.0.8 `COCOeval` (keypoints, COCO-17 sigmas, GT area, maxDets 20) after shared rescoring, with annotation id 0 kept as in the published numbers. The test0 score rank is **TBD** for arm B and 51 for the released model (D13).
- *Selection:* only on Val400, and pre-registered at `72f9449` before the single Test100 evaluation. No Test100 result fed back into any choice.

6 Analysis

6.1 Where AP is lost, and how Val400 relates to Test100

Per threshold. Figure 5 (left) plots Test100 AP at each OKS threshold, together with the `paper`-protocol ceiling of a perfect model (Section 3.4). Every model sits on the ceiling up to $t = 0.75$. They separate only above it: the released model's $AP_{80:95}$ is 86.2. On this test set, AP is decided by localisation precision, not by gross failures.

Val400 against Test100. Figure 5 (right) shows Val400 against Test100 AP. Across the 5 runs with both, the Spearman rank correlation is 0.70, and Test100 exceeds Val400 by +4.7 AP on average. Test100's looser boxes explain part of that gap (Section 4.1). The ranking transfers reasonably even though Test100 was chosen for complex poses.

Evaluation choices. Table 13 (Appendix J) holds the released model's predictions fixed and changes one evaluation choice at a time. Each choice moves AP by about as much as FiDIP's published gain over fine-tuning (0.9):

- the matching protocol: +1.4 AP. Moving test0's score rank alone swings the `paper` AP from 92.98 to 94.07;
- Test100-clean: −0.8;
- validate500's boxes: 92.0 versus 93.6;
- the README layout's misplaced `test11`: −1.5;
- the placeholder area: 98.9 AP on Val400, against 90.4.

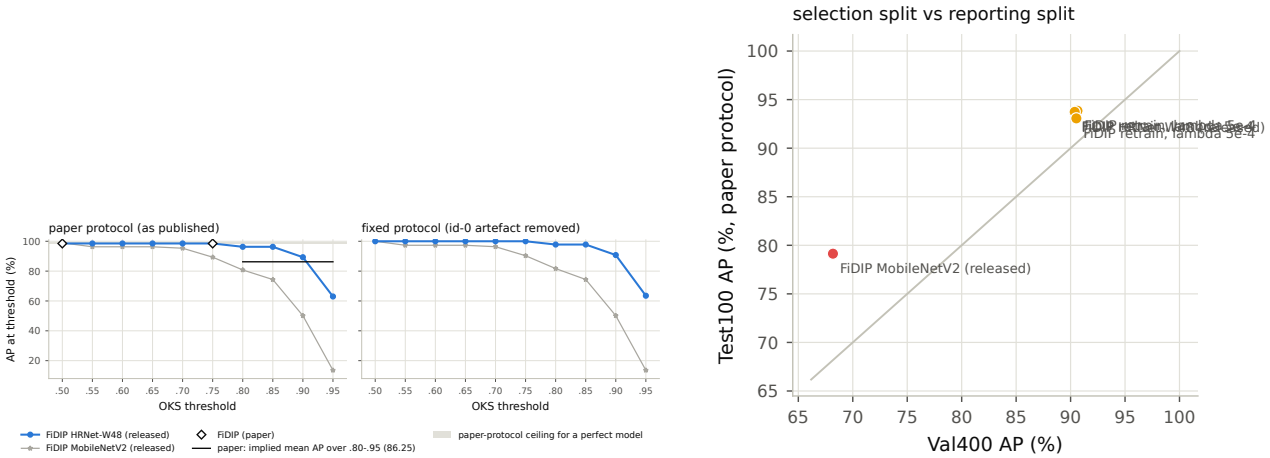


Figure 5: Left: Test100 AP at each OKS threshold (`paper` protocol); the band is the ceiling of a perfect model under the id-0 artifact. Right: Val400 against Test100 AP for every run evaluated on both sets.

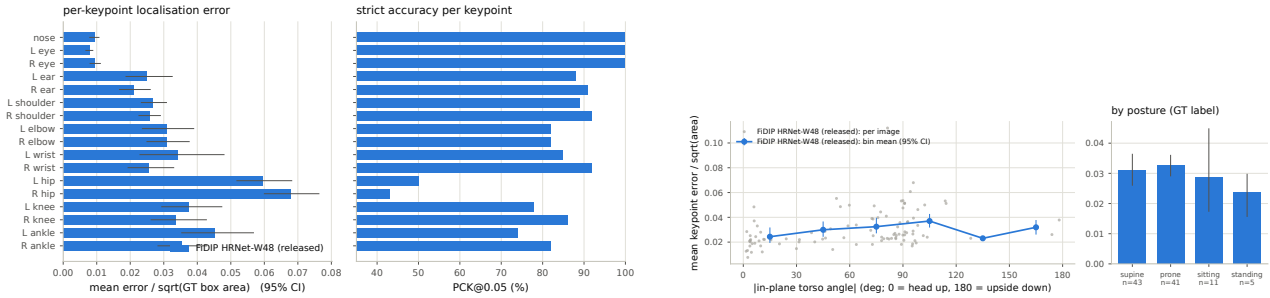


Figure 6: Left: per-keypoint error on Test100, normalised by $\sqrt{wh}$ of the GT box (the OKS scale). Right: error by in-plane torso orientation; sideways infants carry most of the error.

6.2 Per-keypoint accuracy and failure modes

Per keypoint. Figure 6 (left) and Table 9 break the error down by joint. **Hips are by far the hardest joints**: their median error is more than twice that of any other group, and fewer than half fall within 5% of the box scale. This fits infant anatomy: hip centres are hidden under diapers and clothing, and legs are often flexed against the trunk. The paper likewise found the lower body hardest for adult-trained models [16]. Heads and wrists are the most precise.

Failure modes. `eval/taxonomy.py` assigns failure modes with unit-tested rules (Table 15 in Appendix J gives counts and exact criteria).

- **Rotation dominates** (Figure 6, right). 65 of the 100 infants lie sideways, and their mean OKS deficit is 5.5%, against 3.0% for upright infants.
- Left/right swaps are rare: 1 image, a back view.
- No prediction is a whole-body mirror.
- 4 images are heavily occluded, and 2 have an inverted torso.

The failures are mostly a matter of degree: sideways and partly hidden infants get somewhat imprecise joints, not catastrophic errors. That is consistent with AP being decided at strict thresholds. Figure 7 shows the worst Test100 cases.

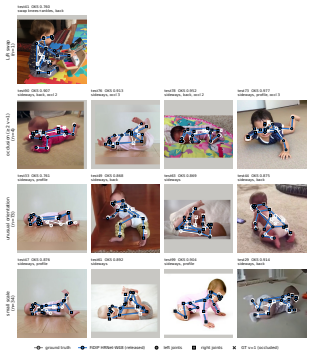


Figure 7: The lowest-OKS Test100 cases for the released FiDIP model, with GT and predictions overlaid (legend in the figure). The best cases are shown in Figure 8.

6.3 Inference demo

Part 6 of the course guide asks for two demos. The command-line demo `tools/demo.py` follows the guide's interface: `--cfg`, `--image`, `--model`, `--output`, and optionally `--conf-threshold` and `--output-size`; it also accepts `--video`. `tools/streamlit_demo.py` lets a user upload an image, set the confidence threshold with a slider, and view keypoints with their confidences. Both were verified on the cluster, the Streamlit app through its AppTest harness and a live health check.

Figure 9 (appendix) shows one practical lesson. The same image is processed with the GT box and with a detector box. Mean keypoint confidence drops from 0.87 to 0.46, so box quality matters as much as the pose model once GT boxes are gone.

7 Limitations

- **A 100-image test set.** About ±0.7 AP is the smallest Test100 difference our paired design resolves, and a single-seed Val400 AP has a CI of about ±1.7. Screening on one seed is therefore coarse, which is why the confirmatory test uses five fresh seeds per arm.
- **GT boxes only.** Like the published work, we evaluate with ground-truth person boxes. With a detector, zero-shot adult models reach only about 42 AP on SyRIP [9], and our demo shows confidences falling sharply with detector boxes (Section 6.3).
- **Selection set.** Val400 is validate500 minus the complex poses, with tighter boxes than Test100. It may favour models that are good on easier poses, although the rank correlation (Section 6.1) suggests the ranking transfers.
- **Reproduction scope.** Our retrains select epochs on Val400, whereas the released loop selects on Test100. They also train on the official labels, while the README points to a mislabelled file. The paper's recipe was run once. The "no gain from adaptation" result holds for the released recipe at three seeds. A gain below the resolution limit cannot be excluded.
- **Kept artifacts.** For comparability, headline numbers keep the pycocotools id-0 artifact and the five Test100 copies of training images. Hence the `fixed` and Test100-clean numbers, and the rule that claims must hold under both protocols.
- **Search scope.** We screened ViTPose-L/H at two input sizes plus a few recipe changes, with COCO-only pose pre-training. Only comparisons within MMPose are controlled, because upstream and MMPose also differ in augmentation and training code.
- **Expanded SyRIP.** Its 400 extra real training images are most likely validate500 \ validate100, i.e. our Val400. Using them would consume the selection set, so this is left for future work with

cross-validated selection.

- **Agent-executed work.** An AI agent did most of the execution and made mistakes (Section 9). Tests, cross-checks, pre-registration and provenance records reduce this risk but cannot remove it.

8 Lessons learned

- **At $n = 100$, evaluation details are as large as method effects.** Five details each move AP by about FiDIP's published gain over fine-tuning (Table 13):
  - the id-0 artifact;
  - the evaluator library;
  - the GT boxes;
  - one misplaced test image;
  - the train/test overlap.

  Building, testing and cross-checking the evaluator *before* running experiments was the most valuable step of the project.
- **Verify the data, not just the code.** Hashing, near-duplicate search and label/image checks found six issues (Table 3). Each would have biased the results silently, and none shows up in a training curve.
- **Read the code, not only the paper.** Run literally, the README trains plain fine-tuning from random weights and selects on the test set. Once those were fixed, the training pipeline reproduced 93.6, but the domain-adaptation gain did not reappear.
- **Read published numbers diagnostically.** $AP_{50} = AP_{75} = 98.5$ alone shows that all headroom is at strict thresholds.
- **Cluster hygiene.** Several failures came from the infrastructure rather than the research:
  - a slow `du` hung an SSH probe;
  - the login node killed `unzip`;
  - reading JPEG headers dropped the session;
  - macOS `openrsync` could not talk to the cluster's rsync 3.2.5;
  - the lab share was unreadable.

  The fixes were `timeout` and BatchMode SSH, Slurm jobs even for light data work, tar over ssh, and our own checksummed download.
- **Guardrails need tests too.** The pre-registration gate parses `PREREGISTRATION.md`. When the selection evidence was once written as a table, three screening runs briefly became eligible; a post-commit check caught this before any evaluation. Configs are typed as well: `TRAIN.LAMBDA 0` (an int) failed, and `0.0` is required.
- **Agents get interrupted.** Workers stopped twice on usage limits, and parallel workers raced on one shared filesystem. Scheduled heartbeats, file-count checks and the live notebooks made recovery cheap, because nothing lived only in an agent's context.

8.1 Reflection: how the agentic workflow changed the approach

Starting out. I began by asking the agent for "maximum points + bonus", and within minutes I interrupted with "Wait" to switch to plan mode. That pause shaped the project: the agent explored the cluster, the code and the data read-only, and I approved a written plan before anything changed. I supplied the course slides and the STUDENT_GUIDE, which the agent could not find on its own. They added deliverables mid-stream.

What the agent was good at. It was strongest at work that is tedious by hand:

- reading unfamiliar code and spotting what decides a reproduction, such as random initialisation, $\lambda = 0$ and test-set selection;
- auditing data files, which exposed the placeholder areas, the mislabelled file, the train/test copies and the id-0 artifact;
- running four streams in parallel, with tests and notes;
- checking citations.

Because checking was cheap, far more of the project went into evaluation conditions than I expected. That is how we learned that a stricter protocol reproduces 93.6 but not the domain-adaptation gain behind it.

Where I stepped in. I forced planning before execution, provided the course material, approved the plan and the parallel setup, set policy (secrets handling, no AI co-author credit), and reviewed the results and this report. The agent's mistakes surfaced through its own checks, not by luck. My takeaway is that an agent speeds up execution most when the human first fixes the goals, the rules and the standard of evidence.

9 AI-tool use statement

Tool. The project used Claude Code [5], Anthropic's agentic coding tool, with the Claude Opus 5.5 model. It ran on the student's laptop with passwordless SSH to Explorer, as one orchestrator plus four worker sub-agents (repro, improve, analysis, report) briefed in writing, and a fresh-context reviewer at the end. The AI is credited as a tool, not as an author. The student is responsible for the report and the repository.

What the agent did.

- planned in plan mode, then executed the approved plan;
- wrote the code: data scripts, Slurm jobs, configs, the evaluator, tests and statistics, demos, figure and table generators, and this report's source;
- submitted and monitored jobs, and diagnosed their failures;
- audited the upstream code and data;
- verified the literature;
- kept `SKILL.md`, `AGENT.md`, `MANIFEST.md` and the notebooks current.

What the human did.

- set the goal and constraints (the rubric and the course rules);
- required and approved a plan;
- supplied the course slides and guide;
- asked for parallel agents;
- set the secrets and no-co-author policies;
- reviewed results and documents.

How outputs were verified.

(1) 69 unit tests, run locally and on the cluster.

(2) Agreement with upstream's printed AP on 21 run/split pairs.
(3) SHA256 for every download and checkpoint, plus a `run.json` provenance record per run.
(4) Script-enforced pre-registration.
(5) No hand-typed results: every number comes from `report/generated/`. Numbers without a generated key sit in `manual_numbers.tex` with their source, mirrored in `results/tables/provenance.csv`.
(6) Online verification of every reference (`report/check_bib.py`).
(7) A fresh-context final review.

Agent errors caught by these checks.

- estimated timestamps;
- a wrong literature claim about resizing position embeddings;
- a hung SSH probe;
- a shared-filesystem race;
- the pre-registration parse slip;
- `TRAIN.LAMBDA 0` instead of `0.0`.

Integrity. No other student's repository was opened or used. The files the guide references but upstream lacks were written from the guide's documented interface.

References

[1] COCO API. https://github.com/cocodataset/cocoapi. Matlab, Python (`pycocotools`) and Lua APIs for the COCO dataset; GitHub repository. Accessed 2026-11-05.

[2] Infant-Pose-Estimation: Official code for "invariant representation learning for infant pose estimation with small data" (FG 2021). https://github.com/ostadabbas/Infant-Pose-Estimation. GitHub repository. Accessed 2026-11-05.

[3] Synthetic and real infant pose (SyRIP) dataset. https://coe.northeastern.edu/Research/AClab/SyRIP/. Dataset download page. Accessed 2026-11-05.

[4] xtcocoapi: Extended COCO-API (`xtcocotools`). https://github.com/jin-s13/xtcocoapi. Version 1.14.3 on PyPI, used by MMPose 1.3.2. Accessed 2026-11-05.

[5] Anthropic. Claude Code. https://code.claude.com/docs/en/overview, 2026. Agentic coding tool; product page https://claude.com/product/claude-code. Accessed 2026-11-05.

[6] S. Bose, H. Dela Cruz, A. Dutta, E. Kokkoni, K. Karydis, and A. K. Roy-Chowdhury. Leveraging synthetic adult datasets for unsupervised infant pose estimation. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 5562–5571, Nashville, TN, USA, June 2025. IEEE. doi:10.1109/cvprw67362.2025.00553.

[7] X. Cao, X. Li, L. Ma, Y. Huang, X. Feng, Z. Chen, H. Zeng, and J. Cao. AggPose: Deep aggregation vision transformer for infant pose estimation. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence (IJCAI-22)*, pages 5045–5051, Vienna, Austria, July 2022. International Joint Conferences on Artificial Intelligence Organization. doi:10.24963/ijcai.2022/700. Special Track on AI for Good.

[8] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993. ISBN 978-0-412-04231-7. doi:10.1201/9780429246593.

[9] F. Gama, M. Mísař, L. Navara, S. T. Popescu, and M. Hoffmann. Automatic infant 2D pose estimation from videos: Comparing seven deep neural network methods. *Behavior Research Methods*, 57(10):280, 2025. doi:10.3758/s13428-025-02816-x.

[10] Y. Ganin and V. Lempitsky. Unsupervised domain adaptation by backpropagation. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1180–1189, Lille, France, July 2015. PMLR. URL https://proceedings.mlr.press/v37/ganin15.html.

[11] Y. Ganin, E. Ustinova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. Marchand, and V. Lempitsky. Domain-adversarial training of neural networks. *Journal of Machine Learning Research*, 17(59): 1–35, 2016. URL https://jmlr.org/papers/v17/15-239.html.

[12] A. Grafton, J. M. Warnecke, M. Li, E. He, L. Thomson, K. Beardsall, and J. Lasenby. Neonatal pose estimation in the unaltered clinical environment with fusion of RGB, depth and IR images. *npj Digital Medicine*, 8(1):539, 2025. doi:10.1038/s41746-025-01929-z.

[13] N. Hesse, S. Pujades, J. Romero, M. J. Black, C. Bodensteiner, M. Arens, U. G. Hofmann, U. Tacke, M. Hadders-Algra, R. Weinberger, W. Müller-Felber, and A. S. Schroeder. Learning an infant body model from RGB-D data for accurate full body motion analysis. In *Medical Image Computing and Computer Assisted Intervention – MICCAI 2018: 21st International Conference, Granada, Spain, September 16–20, 2018, Proceedings, Part I*, volume 11070 of *Lecture Notes in Computer Science*, pages 792–800, Cham, 2018. Springer. doi:10.1007/978-3-030-00928-1_89.

[14] N. Hesse, C. Bodensteiner, M. Arens, U. G. Hofmann, R. Weinberger, and A. S. Schroeder. Computer vision for medical infant motion analysis: State of the art and RGB-D data set. In *Computer Vision – ECCV 2018 Workshops: Munich, Germany, September 8–14, 2018, Proceedings, Part VI*, volume 11134 of *Lecture Notes in Computer Science*, pages 32–49, Cham, 2019. Springer. doi:10.1007/978-3-030-11024-6_3.

[15] J. Huang, Z. Zhu, F. Guo, and G. Huang. The devil is in the details: Delving into unbiased data processing for human pose estimation. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5699–5708, Seattle, WA, USA, June 2020. IEEE. doi:10.1109/cvpr42600.2020.00574.

[16] X. Huang, N. Fu, S. Liu, and S. Ostadabbas. Invariant representation learning for infant pose estimation with small data. In *2021 16th IEEE International Conference on Automatic Face and Gesture Recognition (FG 2021)*, pages 1–8, Jodhpur, India, Dec. 2021. IEEE. doi:10.1109/fg52635.2021.9666956.

[17] X. Huang, M. Wan, L. Luan, B. Tunik, and S. Ostadabbas. Computer vision to the rescue: Infant postural symmetry estimation from incongruent annotations. In *2023 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 1909–1917, Waikoloa, HI, USA, Jan. 2023. IEEE. doi:10.1109/wacv56688.2023.00195.

[18] Z. Hui, J. Wang, X. Dong, G. Zhang, X. Wang, J. Peng, Q. Liu, X. Yuan, Y. Zhang, J. O. Yin, W. Ma, Q. Lyu, S. N. Lupolt, K. E. Nachman, A. Kortylewski, and A. Yuille. InfantNet: A large scale dataset for infant body pose and shape estimation. arXiv preprint arXiv:2610.40467, 2026. URL https://arxiv.org/abs/2610.40467.

[19] L. Jahn, S. Flügge, D. Zhang, L. Poustka, S. Bölte, F. Wörgötter, P. B. Marschik, and T. Kulvicius. Comparison of marker-less 2D image-based methods for infant pose estimation. *Scientific Reports*, 15(1): 12148, 2025. doi:10.1038/s41598-025-96206-0.

[20] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: Common objects in context. In *Computer Vision – ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6–12, 2014, Proceedings, Part V*, volume 8693 of *Lecture Notes in Computer Science*, pages 740–755, Cham, 2014. Springer. doi:10.1007/978-3-319-10602-1_48.

[21] MMPose Contributors. OpenMMLab pose estimation toolbox and benchmark. https://github.com/open-mmlab/mmpose, 2020.

[22] Northeastern University Research Computing. NURC RTD: HPC documentation for the Northeastern University Explorer Cluster. https://rc-docs.northeastern.edu/en/latest/, 2026. Accessed 2026-11-05.

[23] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4510–4520, Salt Lake City, UT, June 2018. IEEE. doi:10.1109/cvpr.2018.00474.

[24] K. Sun, B. Xiao, D. Liu, and J. Wang. Deep high-resolution representation learning for human pose estimation. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5686–5696, Long Beach, CA, USA, June 2019. IEEE. doi:10.1109/cvpr.2019.00584.

[25] G. Varol, J. Romero, X. Martin, N. Mahmood, M. J. Black, I. Laptev, and C. Schmid. Learning from synthetic humans. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4627–4635, Honolulu, HI, July 2017. IEEE. doi:10.1109/cvpr.2017.492.

[26] B. Xiao, H. Wu, and Y. Wei. Simple baselines for human pose estimation and tracking. In *Computer Vision – ECCV 2018: 15th European Conference, Munich, Germany, September 8–14, 2018, Proceedings, Part VI*, volume 11210 of *Lecture Notes in Computer Science*, pages 472–487, Cham, 2018. Springer. doi:10.1007/978-3-030-01231-1_29.

[27] Y. Xu, J. Zhang, Q. Zhang, and D. Tao. ViTPose: Simple vision transformer baselines for human pose estimation. In *Advances in Neural Information Processing Systems*, volume 35, pages 38571–38584. Curran Associates, Inc., 2022. doi:10.52202/068431-2795. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/fbb10d319d44f8c3b4720873e4177c65-Abstract-Conference.html.

[28] F. Zhang, X. Zhu, H. Dai, M. Ye, and C. Zhu. Distribution-aware coordinate representation for human pose estimation. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7091–7100, Seattle, WA, USA, June 2020. IEEE. doi:10.1109/cvpr42600.2020.00712.

[29] Z. Zhou, Z. Jiang, W. Chai, C.-Y. Yang, L. Li, and J.-N. Hwang. Efficient domain adaptation via generative prior for 3D infant pose estimation. In *2024 IEEE/CVF Winter Conference on Applications of Computer Vision Workshops (WACVW)*, pages 51–59, Waikoloa, HI, USA, Jan. 2024. IEEE. doi:10.1109/wacvw60836.2024.00013.

A Reproduction commands

All commands run from the repository root. Cluster commands run in `/scratch/purohitham.v/fidip/repo`, which is kept in sync with `scripts/sync_to_cluster.sh`. `README.md` has the full, current sequence, and each run's exact command is recorded in its `run.json` and in `MANIFEST.md`.

```
# 0. access (SKILL.md): passwordless key, host alias "explorer"; never compute on the login node
ssh -o BatchMode=yes -o ConnectTimeout=15 explorer 'hostname'
bash scripts/sync_to_cluster.sh # tar-over-ssh push of code (never deletes)
# 1. environments (CPU jobs)
sbatch scripts/slurm/build_env_fidip.sbatch # pip-only py3.12 / torch 2.3.1 cu121
sbatch scripts/slurm/build_env_mmpose.sbatch # py3.10 / torch 2.1.2 / mmpose 1.3.2
sbatch scripts/slurm/build_env_analysis.sbatch # numpy 1.26.4 + pycocotools 2.0.8
# 2. data: download, SHA256 verify, unzip on a compute node, upstream layout, split files
sbatch scripts/slurm/get_data.sbatch
# 3. checkpoints (SHA256 in MANIFEST.md section 4)
sbatch scripts/slurm/get_ckpts_mmpose.sbatch; sbatch scripts/slurm/prep_ckpts_mmpose.sbatch
# 4. FiDIP retrain = the README command plus the two overrides it needs (one seed shown)
python tools/train_adaptive_model_hrnet.py \
  --cfg experiments/coco/hrnet/w48_384x288_adam_lr1e-3_infant.yaml \
  TRAIN.CHECKPOINT models/coco/w48_384x288.pth TRAIN.LAMBDA 0.0005 # 0.0 = fine-tune ablation
# 5. MMPose run (Val400 selection inside; never touches Test100)
sbatch --export=ALL,RUN_ID=<RUN_ID>,CONFIG=mmpose_exps/configs/<config>.py,SEED=1 \
  scripts/slurm/train_mmpose.sbatch
# 6. final Test100 evaluation of a pre-registered run (refuses otherwise)
sbatch --export=ALL,RUN_ID=<RUN_ID> scripts/slurm/final_test_eval_mmpose.sbatch
# 7. metrics, CIs, tables, figures, report
python eval/eval_coco.py --gt <split>.json --pred runs/<RUN_ID>/preds_test100.json \
  --out runs/<RUN_ID>/metrics_test100.json --per-image runs/<RUN_ID>/per_image_test100.json
python eval/bootstrap_ci.py multiseed --gt <test100>.json --a <A_s1> <A_s2> <A_s3> --b <B> --out ci.
    json
bash scripts/sync_results.sh && python scripts/collect_results.py # -> report/generated/*.tex
bash report/build.sh # -> report/FiDIP_Report.pdf
```

B Upstream traps and patches

Six details of the upstream code matter for anyone reproducing it:

(i) The literal README command is plain fine-tuning from random weights (Section 2.3). We pass `TRAIN.CHECKPOINT` and `TRAIN.LAMBDA` on the command line rather than editing code.

(ii) The loader always builds the adult `train_pre_infant` set, even when `PRE_EPOCH` is 0.

(iii) `lib/dataset/syrip.py` skips evaluation for any split whose name contains "test".

(iv) `tools/test_adaptive_model.py` loads weights with `strict=False`, so a key-prefix mismatch would silently leave the network at its initialisation. We added a strict-load check.

(v) The compiled NMS extensions are never used, so `make` can be skipped.

(vi) For ViTPose at 384×288, MMPretrain infers a *square* grid from the 192 position embeddings of the 16×12 COCO checkpoint and fails. We resized them offline (bicubic, 16×12 → 24×18, `mmpose_exps/tools/resize_vit_pos_embed.py`).

Table 7: Patches to the vendored upstream code (`third_party/Infant-Pose-Estimation`), one commit each, generated from `MANIFEST.md` §6.

Patch	Commit	Change	Why
P1	`7168b1e`	compiled-NMS imports made optional	only the pure-Python OKS-NMS is used; no `make`/nvcc needed
P3	`4f3f869`	`torch.load(map_location, weights_only=False)`; strict loading at test time	prevents a silent `strict=False` misload
P4	`7ec379a`	build the `train_pre` loader only if `PRE_EPOCH` > 0	the shipped code always required its JSON
P5	`67fde20`	`is '*'` $\rightarrow$ `== '*'`	SyntaxWarning under Python 3.12
P6	`763a36f`	`mkdir(parents=True, exist_ok=True)`	nested output directories
P7	`25cb420`	loss files written to the run directory; no `add_graph`	no pollution of the working directory; removes a slow CPU trace
P8	`44dcb09`	per-epoch `epochs.csv` (train MSE, Val400 loss, AP/AR)	over/underfitting monitoring (guide Part 4.5)
P10	`fe6d81c`	any split name (`test100`, `val400`, `validate500`); evaluation of labelled `test*` splits	upstream hard-codes split names
P11	`596fb7b`	`SEED` option	repeatable seeds
demo	`7e68cfe`	added `tools/demo.py`, `tools/streamlit_demo.py`, `gpu_test.py`	guide Parts 1.3 and 6; not in upstream

C Evaluator tests

`python -m pytest eval/tests` runs the evaluator's unit tests on mock data; no SyRIP files are needed. They check:

- that the GT used as the prediction gives AP = AR = 1;
- the id-0 artifact (`paper` versus `fixed`) and the exact `paper`-protocol ceiling formula;
- monotone degradation of AP, OKS and PCK under increasing keypoint noise;
- bit-identical metrics across twelve prediction encodings;
- that unknown images are dropped and missing predictions count as misses;
- the rescoring invariances: identical results for files already scored by the rule, and indifference to random file scores, monotone score transforms and confidence scaling;
- that the rescoring rule equals upstream's loop;
- the constant-area guard;
- that per-image OKS equals `COCOeval`'s internal values;
- exact per-keypoint errors and PCK on constructed shifts;
- that per-threshold AP averages back to AP, AP_{50} and AP_{75};
- that OKS-NMS keep-lists equal upstream `nms.py` on random instance sets;
- the command line end to end.

The bootstrap, ensemble, taxonomy and analysis tools have their own tests in the same folder.

Table 8: Resolved training and inference settings. Upstream values come from the released yaml and code at `a76b4f4`, plus the two command-line overrides. MMPose values come from the dumped configs of the selected runs.

Setting	FiDIP retrain (upstream code)	Arm A: HRNet-W48 (MMPose)	Arm B: ViTPose-H (MMPose)
Initial weights	COCO DarkPose `w48_384x288.pth`	MMPose HRNet-W48 DARK COCO (`39c3c381`)	MMPose ViTPose-H COCO 256×192 (`e32adcd4`)
Input / codec	384×288, heatmap 96×72, $\sigma = 3$, DARK	384×288, DARK	256×192, UDP
Optimiser	Adam, LR 10^{-4} constant, no weight decay	Adam, LR 10^{-4}	AdamW, LR 10^{-4}, wd 0.1, layer decay 0.85, clip 1.0, drop-path 0.55
Batch / epochs	20 / 20	32 / 30	32 / 30
Schedule	constant (steps at 40, 200 never reached)	warm-up, ×0.1 at epochs 24 and 29	same as arm A
Domain adaptation	$\lambda = 5{\cdot}10^{-4}$ (FiDIP) or 0.0; pre-stage 10 epochs on the adult set; D: Adam 10^{-4}	none	none
Frozen layers	first 109 parameter tensors	none	none
Augmentation	flip 0.5; rotation $\mathcal{N}(0, 45°)$ clipped to ±90° w.p. 0.6; scale ±35%; half-body 0.3	MMPose COCO default	same as arm A
Real : synthetic	balanced sampler 1:1 (real oversampled ×5)	1:1 (real ×5 per epoch)	same
Test time	GT boxes, flip, DARK (blur kernel 11)	GT boxes, flip	GT boxes, flip
Selection	best Val400 AP epoch	best Val400 AP epoch	best Val400 AP epoch
Seeds	s1–s3	s2–s6 (R131–R135)	s2–s6 (R136–R140)

D Hyperparameters

E Per-keypoint results

F Compute used

Every run's Slurm job ID, partition, GPU model, start and end times are recorded in `results/registry.csv` and in its `run.json`, so the compute behind any number can be read back with `sacct -j <job>`. GPU jobs ran on H200, A100 (40/80 GB), L40S and V100 nodes of the `gpu` and `sharing` partitions, with several runs packed per job where memory allowed (D14, D22). Data preparation, environment builds and analysis ran as CPU jobs. Per-user quotas (four running `gpu` jobs, two on `sharing`) and two agent usage-limit stops bounded the number of seeds more than the GPU time did.

G Provenance: from a number to its run

To trace any number in this report:

1. `MANIFEST.md` §8 maps each reported number to a run ID and a metrics file.
2. The run's row in `results/registry.csv` gives the Slurm job ID, config, seed, GPU, git SHA and checkpoint SHA256.

Table 9: Per-keypoint accuracy on Test100: median error as a percentage of $\sqrt{wh}$ of the GT box, and PCK with the same normaliser, over labelled joints only.

	Released FiDIP (R001)			Best configuration		
Keypoint	med. err. (%)	PCK@.05	PCK@.1	med. err. (%)	PCK@.05	PCK@.1
Head (nose, eyes, ears)	0.8	95.8	98.6	**TBD**	**TBD**	**TBD**
Shoulders	2.3	90.5	99.5	**TBD**	**TBD**	**TBD**
Elbows	1.7	82.0	94.0	**TBD**	**TBD**	**TBD**
Wrists	1.3	88.5	92.5	**TBD**	**TBD**	**TBD**
Hips	5.3	46.5	83.0	**TBD**	**TBD**	**TBD**
Knees	2.2	82.0	94.5	**TBD**	**TBD**	**TBD**
Ankles	2.4	78.0	92.5	**TBD**	**TBD**	**TBD**
All	1.8	83.2	94.4	**TBD**	**TBD**	**TBD**

3. The run directory `runs/<RUN_ID>/` on `/scratch` (with persistent copies in `~/fidip_artifacts/`) holds `run.json`, the logs, the predictions and the `eval_coco.py` output. Its `gt.sha256` and `pred.sha256` fields pin the exact files.
4. `report/generated/*.tex` is regenerated from those metrics by `scripts/collect_results.py`; the report only reads it with `\input`.

Decisions (D1, D2, ...) are dated and explained in `AGENT.md`, and command-level detail is in `notebook/<stream>.md`.

H Rubric traceability

Table 10: Where each grading item is addressed.

Rubric item	Report	Repository
Environment setup, data preparation	Sections 3.2 and 3.3	`envs/`, `scripts/get_data.sh`, `scripts/make_splits.py`
Working inference and evaluation pipeline	Sections 3.4, 4.1 and 6.3	`eval/eval_coco.py`, `eval/tests/`, `tools/demo.py`
Reproducible instructions, code, configs	Appendices A, D and G	`README.md`, `MANIFEST.md`, `scripts/slurm/`, `mmpose_exps/configs/`
Correct quantitative evaluation	Sections 3.4 and 6.1	`eval/`, `results/PREREGISTRATION.md`
Comparison with the published baseline	Tables 2, 4 and 5; box in Section 5.4	`results/registry.csv`
Visualisations, success/failure analysis	Section 6	`eval/visualize.py`, `eval/analyze.py`, `eval/taxonomy.py`
Organisation; methods and choices	Sections 1 to 3 and 5.1	`README.md`
Limitations, lessons, references, AI use	Sections 7 to 9; references	`AGENT.md`, `report/refs.bib`
Bonus: improvement, ablation, repeats, conditions	Section 5; Tables 5, 11, 12 and 14	`results/PREREGISTRATION.md`, `MANIFEST.md`
Live `SKILL.md` / `AGENT.md` / `MANIFEST.md`	Sections 3.1 and 9	repository root

I Implementation details

I.1 Compute and agentic workflow

Compute. Training and evaluation ran under Slurm on Northeastern's Explorer cluster [22]:

- `gpu` (H200/A100, 8 h limit) for training;
- `sharing` (1 h) for smoke tests and evaluations;
- `short` for CPU jobs.

Datasets, environments and checkpoints stay on `/scratch`; only code, metrics, predictions and figures travel to the laptop, which had about 7 GB free.

Agents. Claude Code [5], running Claude Opus 5.5, acted as an *orchestrator* (Figure 2). After the student approved a written plan, it launched four parallel workers, each with a self-contained brief:

- *repro*: data, environments, released checkpoints and retraining;
- *improve*: MMPose experiments;
- *analysis*: the evaluator, statistics, figures and generated tables;
- *report*: this document and the README.

A fresh-context reviewer checks the final state against the rubric.

Live files. The orchestrator owns three of them:

- `SKILL.md`: how to operate the cluster;
- `AGENT.md`: the lab notebook of asks, attempts, failures and dated decisions D1, D2, ...;
- `MANIFEST.md`: datasets, checkpoints, configs, and the run behind every number.

Workers log commands, job IDs and errors in `notebook/<stream>.md`, and register runs in `results/registry.csv`.

Shared rules. All agents load `CLAUDE.md`, which requires:

- no compute on login nodes;
- no selection on Test100;
- one evaluator for every number;
- secrets only in a git-ignored `.env`;
- timestamps from `date`.

Cluster loop. The agent connects with `ssh -o BatchMode=yes -o ConnectTimeout=15` and wraps slow login-node commands in `timeout`, so failures surface quickly. For each job it:

1. pushes code with tar over ssh;
2. submits an `sbatch` script, smoke-tested first on `sharing`;
3. polls `squeue`, `sacct` and the logs;
4. pulls back only small artifacts.

Each run's directory `runs/<RUN_ID>/` holds its resolved config, logs, predictions, evaluator outputs, and a `run.json` with the commit and tree SHAs, Slurm job ID, node, GPU, seed and checkpoint SHA256.

I.2 Environments

Environments. The upstream `fidip_env.yml` cannot be built: its pip stage pins `grpcio==1.48.2`, which has no Python 3.12 wheel, and `mkl-*` packages with no wheels. We built three environments instead:

- *FiDIP:* pip-only, Python 3.12, PyTorch 2.3.1 (CUDA 12.1), NumPy 1.26.4, with every imported package pinned to the yml's version.
- *MMPose:* Python 3.10, PyTorch 2.1.2, MMEngine 0.10.7, MMCV 2.1.0, MMPretrain 1.2.0 and MMPose 1.3.2 [21], from conda-forge, with `numpy<2` and `opencv-python<4.12`.
- *Analysis:* CPU-only, with the same NumPy and pycocotools pins as FiDIP.

Package lists are in `envs/` and checkpoint hashes in `MANIFEST.md`.

Upstream patches. Appendix B lists six upstream traps and our patches. The most consequential trap, the README training command, needs no code change: `TRAIN.CHECKPOINT` and `TRAIN.LAMBDA` are passed on the command line. The course guide's `gpu_test.py`, `tools/demo.py` and `tools/streamlit_demo.py` (and `fidip_simple.yml`) do not exist upstream. We wrote the GPU check and both demos from the guide's documented interface.

I.3 Data preparation

Download. A Slurm job (`scripts/get_data.sh`):

1. downloads both public SyRIP archives, which needed no password;
2. verifies their SHA256;
3. extracts them on a compute node, because the login node killed `unzip`;
4. checks file counts against the archives;
5. builds the upstream layout from symlinks and writes the split files.

The adult `train_pre_infant` set (2029 real and 2028 synthetic images), described upstream as available on request, ships inside `syrip_for_train.zip`, so the classifier pre-stage can be run faithfully. `SyRIP.zip`'s `images/train_infant/` also holds 300 unannotated real images, 199 of them byte-identical to validate500 images. We use only images listed in an annotation file.

Training labels. Two annotation files cover the 1200 training images: the official `200R_1000S` file in `SyRIP.zip`, and `train_infant.json` in `syrip_for_train.zip`. They agree on the real images but differ on every synthetic one, because the second file carries MINI-RGBD labels:

- its source fields point to MINI-RGBD frames;
- eyes and ears are zeroed on most entries;
- some sizes and boxes do not fit the renders, with a median keypoint offset of 191 px from the official labels (`notebook/repro.md`).

The images in the two archives are identical, except that the second archive's `validate_infant/test11.jpg` is a copy of `test24.jpg`. Following the README literally therefore trains on mislabelled synthetic data and scores one test image against the wrong picture: the released checkpoint scores 92.1 AP on that archive's test images, against 93.6 AP on the correct ones. We use `SyRIP.zip`'s images and `200R_1000S` labels everywhere (D17); attempt R020 measures what the README-literal path costs.

Val400 (D15). validate100 is a subset of validate500 (image ids 0–99, with identical files). **Val400** is validate500 ids 100–499 minus three images that duplicate Test100 images (two exact copies, one near-duplicate), which leaves 397 images. Exact copies were found by file hash, and near-duplicates by a flip-aware perceptual hash with visual confirmation (`scripts/near_dups.py`).

Train/test overlap (D16). Test100 images `test20`, `test42`, `test46` and `test50` are byte-identical to training images 73, 315, 272 and 340, and `test7` is a near-duplicate of training image 47. We keep Test100 as shipped, for comparability with the published numbers (which carry the same overlap). As a sensitivity check we also report the 95-image *Test100-clean* subset, where the released model's AP changes by −0.8. Two further properties of the shipped GT need care.

Placeholder areas (D8). Every validate500 annotation carries the same `area`, 2443298.48 ($\sqrt{\text{area}} \approx 1563$ px), and the training file carries another constant, 14232.77; validate100 instead uses area $= w \cdot h$ of its box. Since OKS divides squared errors by the area (Equation (1)), the shipped Val400 makes almost any prediction look perfect: the released model scores 98.9 AP on it. We rebuilt Val400, validate500 and the framework-neutral training file with area $= w \cdot h$ from their own boxes, and left Test100 exactly as shipped. `scripts/make_splits.py` asserts this rule, and the evaluator refuses GT files whose areas are all identical.

Different boxes. On the 100 shared images, validate100's boxes are a median 12% larger than validate500's, while the keypoints agree on 98 images; the other two were re-annotated. Val400 boxes are therefore tighter than Test100's, which means a smaller OKS scale and tighter crops (Section 6.1).

Protocol (D1). Figure 3 summarises the protocol:

- Every choice of epoch, hyperparameter, backbone or recipe is made on Val400.
- A configuration reaches Test100 only after a commit lists it, with its Val400 evidence and checkpoint rule, in `results/PREREGISTRATION.md`.
- The final-evaluation script refuses unregistered or repeated runs, and checks the checkpoint SHA256 against `run.json`.
- The released checkpoints and the official-recipe retrains are reproduction targets, so they are evaluated on Test100 directly; the retrains' epochs are still chosen on Val400.

Why Val400. It keeps the training data identical to the paper's, and it is almost twice as large as the real training set; cross-validation on the 200 real images could not match that without shrinking training. It has two costs: Test500 is no longer clean, so we never report it, and Val400 is not the complex-pose subset, so it may rank models differently from Test100 (Section 6.1).

J All attempts, screening, sensitivity and failure counts

K Demo output

Table 11: Backbone and input-size screening under one MMPose recipe (Val400, seed 1). Test100 was not evaluated for screening runs.

Backbone	Input	Params (M)	Val400 AP	Val400 $AP_{80:95}$	Seeds	Test100 AP	GPU-h/run
HRNet-W48 + DARK (control)	384×288	**TBD**	91.5	81.6	1	—	**TBD**
ViTPose-L	256×192	**TBD**	92.8	84.8	1	—	**TBD**
ViTPose-L	384×288	**TBD**	92.8	84.8	1	—	**TBD**
ViTPose-H	384×288	**TBD**	93.3	85.8	1	—	**TBD**

Table 12: One-factor ablations on Val400 (one seed). Δ is relative to the stated default.

Factor	Setting	Applied to	Seeds	Val400 AP	Δ
Learning rate	2e-4 (default 1e-4)	HRNet-W48 DARK 384 (MMPose control)	1	91.4	−0.1
Learning rate	2e-4 (default 1e-4)	ViTPose-L 256 (MMPose)	1	92.6	−0.2
Learning rate	5e-5 (default 1e-4)	ViTPose-L 256 (MMPose)	1	92.6	−0.2
FiDIP λ	1e-4	FiDIP retrain, λ 5e-4	1	91.0	+0.5
FiDIP λ	2e-3	FiDIP retrain, λ 5e-4	1	89.7	−0.8
FiDIP λ	1e-2	FiDIP retrain, λ 5e-4	1	85.8	−4.7
Learning rate	5e-5 (default 1e-4)	HRNet-W48 DARK 384 (MMPose control)	1	91.7	+0.2
Training data	real only (200)	ViTPose-L 256 (MMPose)	1	92.2	−0.6
Rotation aug.	up to ±180 deg	ViTPose-L 256 (MMPose)	1	92.6	−0.2

Table 13: Sensitivity of AP to evaluation choices, with predictions held fixed. Headline numbers use the first row of each pair.

Condition	Variant	Split	Released FiDIP	Best config.
Matching protocol (D10, D13)	`paper`: ids as shipped (headline)	Test100	93.6	—
	`fixed`: ids shifted off 0	Test100	95.0	—
Train/test overlap (D16)	Test100 as shipped (100)	Test100	93.6	—
	Test100-clean (95; copies of training images removed)	Test100-clean	92.8	—
Test images (D17)	`SyRIP.zip` (ours)	Test100	93.6	—
	`syrip_for_train.zip` (test11 = copy of test24)	Test100	92.1	—
GT boxes of the same 100 images	validate100 (as shipped)	Test100	93.6	—
	validate500 (area = wh)	Test100	92.0	—
OKS area on Val400 (D8)	shipped constant 2443298.48	Val400	98.9	—
	area = wh (ours)	Val400	90.4	—
Metric code	`eval_coco.py` (pycocotools 2.0.8)	Test100	93.6	—
	MMPose `CocoMetric` (xtcocotools)	Test100	95.0	—

Table 14: Every attempt with its Val400 accuracy and outcome, generated from `results/registry.csv`. Each run ID resolves to `runs/<RUN_ID>/` and to `MANIFEST.md`.

Run ID(s)	Stream	Change vs. control	Seeds	Val400 AP	Δ Val400	Test100 AP	Outcome
R001	repro	upstream hrnet_fidip.pth, upstream test code	1	90.4	—	93.6	reproduction target
R002	repro	upstream mobile_fidip.pth, 224×224	1	68.2	—	79.1	reproduction target
R010–R012	repro	official recipe retrain (COCO DarkPose init, PRE_EPOCH 10, λ 5e-4), Val400 epoch selection	3	90.5 ±0.1	—	93.6 ±0.4	reproduction
R013–R015	repro	same recipe with $\lambda = 0$ (no domain adaptation)	3	TBD	—	—	reproduction
R013	repro	$\lambda = 0$ retrain, first submission	1	TBD	—	—	failed: yacs type error (LAMBDA float vs int); resubmitted
R016	repro (DA)	domain-adaptation weight λ 5e-4 → 1e-4 (1 seed)	1	91.0	+0.5	—	above baseline
R017	repro (DA)	domain-adaptation weight λ 5e-4 → 2e-3 (1 seed)	1	89.7	−0.8	—	not above baseline
R018	repro (DA)	domain-adaptation weight λ 5e-4 → 1e-2 (1 seed)	1	85.8	−4.7	—	not above baseline
R101	improve	FiDIP backbone re-implemented in MMPose, no domain adaptation, shared recipe	1	91.5	—	—	screening control (seed s1; supplementary, imp-4)
R101	improve	test-time: noflip	1	90.9	—	—	screening control (seed s1; supplementary, imp-4)
R102	improve	backbone HRNet-W48 → ViTPose-L (UDP, 256×192), same recipe	1	92.8	+1.3	—	above baseline
R102	improve	test-time: noflip	1	92.3	+0.7	—	above baseline
R103	improve	backbone → ViTPose-L (UDP, 384×288), same recipe	1	92.8	+1.3	—	above baseline
R104	improve	backbone → ViTPose-H (UDP, 384×288), same recipe	1	93.3	+1.8	—	above baseline
R105	improve	backbone → ViTPose-H (UDP, 256×192), same recipe	1	93.2	+1.6	—	selected as arm B (seed s1; supplementary, imp-4)
R106	improve	control with LR 1e-4 → 2e-4	1	91.4	−0.1	—	not above baseline
R107	improve	control with LR 1e-4 → 5e-5	1	91.7	+0.2	—	above baseline
R108	improve	ViTPose-L 256 with LR 1e-4 → 2e-4	1	92.6	−0.2	—	not above baseline
R109	improve	ViTPose-L 256 with LR 1e-4 → 5e-5	1	92.6	−0.2	—	not above baseline
R120	improve	train on the 200 real images only (no synthetic)	1	92.2	−0.6	—	not above baseline
R121	improve	rotation augmentation up to 180 degrees	1	92.6	−0.2	—	not above baseline
R181	improve	no infant training (MMPose COCO checkpoint)	1	86.5	—	—	zero-shot reference
R182	improve	no infant training	1	88.2	—	—	zero-shot reference
R183	improve	no infant training (pos-embed resized to 24×18)	1	86.3	—	—	zero-shot reference
R184	improve	no infant training	1	88.8	—	—	zero-shot reference
R185	improve	no infant training (pos-embed resized to 24×18)	1	87.4	—	—	zero-shot reference
—	improve	pre-registered control, 5 fresh seeds s2-s6	0	TBD	—	—	pre-registered (imp-1)
—	improve	pre-registered treatment: backbone ViTPose-H 256×192, same recipe as arm A, 5 fresh seeds s2-s6	0	TBD	—	—	pre-registered (imp-2)
—	improve	mean of the 5 arm-B predictions (eval/ensemble_preds.py)	0	TBD	—	—	pre-registered (imp-3)

Table 15: Automatic failure taxonomy on Test100 (`eval/taxonomy.py`). Counts are images; mean OKS is taken within each group.

		# images		mean OKS	
Failure mode	Rule	Released	Best	Released	Best
Left/right swap	$\geq$1 L/R pair swapped (OKS gain >0.02, error halved)	1	**TBD**	0.760	**TBD**
Whole-body mirror	mirroring all L/R labels raises OKS	0	**TBD**	—	**TBD**
Heavy occlusion	$\geq$2 GT joints with v=1	4	**TBD**	0.937	**TBD**
Inverted torso	|torso angle| > 135°	2	**TBD**	0.948	**TBD**
Low OKS overall	OKS < 0.85	2	**TBD**	0.760	**TBD**

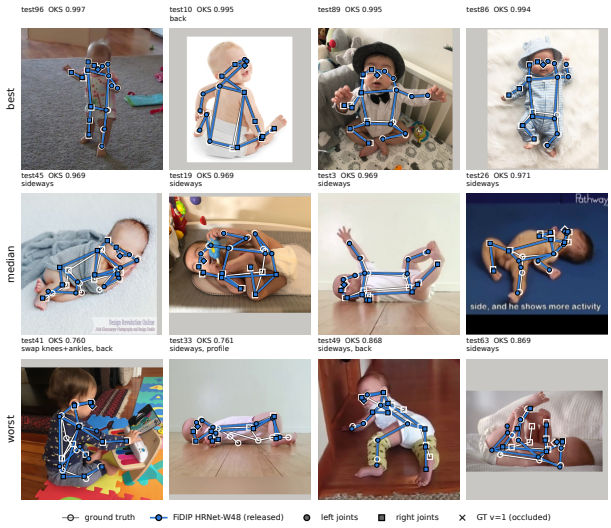


Figure 8: The highest-OKS Test100 cases for the released FiDIP model (legend in the figure).

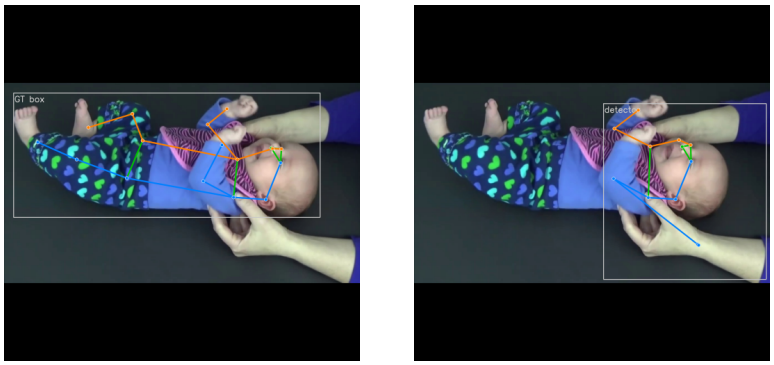


Figure 9: `tools/demo.py` on `test0.jpg`. Left: the GT box. Right: a person-detector box. Confidences are drawn per keypoint.